

ОТЧЕТ

о выполнении практической работы №2.1

Дисциплина «Оптимизация веб-приложений»

Тема: «Разработка паттерна MVC на PHP: маршрутизация запросов»

Специальность: 09.02.07 Информационные системы и программирование

Квалификация: Разработчик веб и мультимедийных приложений

Выполнил: Чеголин Алексей Михайлович

Студент группы: 4ИСИП-923

Проверил:

Преподаватель: Рыбаков А.С.

Дата: «29 сентября 2026 г.»

Оценка: _____

Теоретическая часть:

Model, View, Controller (MVC) — шаблон программирования, разделяющий приложение на три модуля: модель, представление и контроллер. Разделение позволяет изменять каждый компонент независимо от остальных, что упрощает разработку и поддержку веб-приложений.

— Модель (Model) — основная логика и данные: структура, правила обработки данных, доступ к базе.

— Представление (View) — визуализация: получает готовые данные и показывает их пользователю.

— Контроллер (Controller) — обработка действий пользователя: принимает запрос, проверяет данные и передаёт их модели, а затем выбирает представление.

Зависимости компонентов ограничены: модель не зависит от представления и контроллера; представление может читать данные модели, но не менять их; контроллер не отображает данные, но меняет модель по действиям пользователя. Концепцию предложил в конце 1970-х годов сотрудник Херо Херо Трюгве Реексауг, реализация появилась в языке Smalltalk-80, а окончательная версия шаблона была опубликована десять лет спустя в журнале Technology Object.

Удобная физическая аналогия MVC — персональный компьютер: контроллер это клавиатура и мышь, модель — системный блок, где обрабатываются команды и хранятся файлы, представление — монитор. Одна модель может иметь несколько представлений: в Excel одни данные показываются таблицей или круговой диаграммой, а сайт может иметь мобильный и десктопный интерфейс для одних и тех же данных.

Ход работы:

Задание 1. Маршрутизация вручную через switch и файл .htaccess

Создан файл index.php, который разбирает запрошенный адрес и выводит разную страницу в зависимости от пути. Рядом размещён файл .htaccess, направляющий все запросы на index.php, — благодаря ему адреса выглядят как /about, а не /index.php?page=about.

Созданный файл .htaccess:

Строка	Назначение
RewriteEngine On	включает механизм перезаписи адресов
RewriteCond %{REQUEST_FILENAME} !-f	не перенаправлять, если запрошен существующий файл
RewriteCond %{REQUEST_FILENAME} !-d	не перенаправлять, если запрошен существующий каталог

RewriteRule ^(.*)\$ index.php [QSA,L]	все остальные запросы передать index.php, сохранив параметры
---------------------------------------	--

Проверка работы маршрутизации приведена на рисунках 1–3. В верхней части страницы выводится диагностическая информация: какой адрес запрошен, какой путь выделен и каким методом.

```
REQUEST_URI: /
PATH: /
METHOD: GET
```

Главная страница — список статей

Рис. 1 – Главная страница: адрес / — «Главная страница — список статей»

```
REQUEST_URI: /about
PATH: /about
METHOD: GET
```

О проекте

Рис. 2 – Страница /about — «О проекте»

```
REQUEST_URI: /article/5
PATH: /article/5
METHOD: GET
```

Статья №5

Рис. 3 – Страница /article/5 — номер статьи извлечён из адреса

Найденная проблема: код страницы 404 при уже отправленных заголовках

При проверке несуществующих адресов обнаружена ошибка. Страница «404 — страница не найдена» выводится, но HTTP-код ответа остаётся 200, и в браузере

появляется предупреждение. Причина: диагностический блок выводится до разбора маршрута, поэтому к моменту вызова `http_response_code(404)` заголовки уже отправлены и изменить код ответа нельзя.

```
REQUEST_URI: /nope
PATH: /nope
METHOD: GET
```

Warning: `http_response_code()`: Cannot set response code - headers already sent (output started at C:\Users\Aleks\Desktop\Готовые ДЗ\Рыбаков А.С\mvc_demo\stage1-switch\index.php:9) in C:\Users\Aleks\Desktop\Готовые ДЗ\Рыбаков А.С\mvc_demo\stage1-switch\index.php on line 29

404 — страница не найдена

Рис. 4 – Адрес `/nope`: предупреждение о невозможности установить код ответа

```
REQUEST_URI: /article/abc
PATH: /article/abc
METHOD: GET
```

Warning: `http_response_code()`: Cannot set response code - headers already sent (output started at C:\Users\Aleks\Desktop\Готовые ДЗ\Рыбаков А.С\mvc_demo\stage1-switch\index.php:9) in C:\Users\Aleks\Desktop\Готовые ДЗ\Рыбаков А.С\mvc_demo\stage1-switch\index.php on line 29

404 — страница не найдена

Рис. 5 – Адрес /article/abc: номер статьи не является числом, срабатывает та же ошибка

Для поисковых систем это существенно: страница сообщает «404», но отдаёт код 200, то есть ошибка не будет распознана и такой адрес может попасть в индекс. Ошибка исправлена: определение результата разбора выполняется до любого вывода, а диагностический блок печатается уже после установки кода ответа. Проверка исправленного варианта приведена на рисунках 6 и 7.

```
REQUEST_URI: /  
PATH: /  
METHOD: GET  
STATUS: 200
```

Главная страница — список статей

Рис. 6 – Исправленный вариант: страница /about отдаётся с кодом 200

```
REQUEST_URI: /nope
PATH: /nope
METHOD: GET
STATUS: 404
```

404 — страница не найдена

Рис. 7 – Исправленный вариант: адрес /nope отдаётся с кодом 404 без предупреждений

Задание 2. Класс Router

Ручной switch заменён классом Router. Маршруты больше не записаны в коде ветвления, а описываются данными: класс хранит таблицу «метод запроса — шаблон адреса — обработчик». Метасимволы вида {id} превращаются в регулярное выражение, поэтому один маршрут обслуживает все статьи.

Что изменилось по сравнению с первым заданием:

- маршрут задаётся одной строкой в index.php, а не отдельной ветвью switch;
- роутер учитывает метод запроса, поэтому GET и POST на один адрес не конфликтуют;
- появилась автозагрузка классов через spl_autoload_register, подключать файлы вручную не нужно;
- добавление нового раздела не требует правки логики разбора адреса.

Главная — список статей

Рис. 8 – Класс Router: главная страница обслуживается замыканием

О проекте

Рис. 9 – Класс Router: страница /about

Рис. 10 – Класс Router: страница /article/5 с параметром маршрута

A screenshot of a web browser showing a 404 Not Found error. The text "404 Not Found" is visible in the center of the page.

Рис. 11 – Класс Router: несуществующий адрес отдаёт код 404

Контрольные вопросы:

Задание 1

1. Открой /, /about, /article/5, /article/abc, /nore. Куда идут все запросы?

Все запросы идут в один файл index.php: правило в .htaccess перенаправляет туда всё, что не является реально существующим файлом или каталогом. Так реализуется единая точка входа (front controller). Далее адрес разбирается уже внутри index.php: / отдаёт список статей, /about — страницу о проекте, /article/5 — статью с номером 5, а /article/abc и /nore не соответствуют ни одному правилу и обрабатываются ветвью default.

2. Что будет, если маршрутов станет 50? А если добавим POST?

Конструкция `switch` разрастётся на десятки ветвей, станет трудночитаемой и неудобной в поддержке: каждая правка требует изменения одной большой функции, а порядок проверок начинает влиять на поведение. Для `POST` придётся добавлять проверку метода внутри каждой ветви, иначе один и тот же адрес будет обрабатываться одинаково для чтения и для записи, что неверно. Именно поэтому в задании 2 маршруты вынесены в таблицу, а метод запроса стал частью ключа.

Задание 2

1. Что изменилось?

Логика разбора адреса вынесена в отдельный класс `Router`. В `index.php` осталось только описание маршрутов и запуск. Маршруты с параметрами записываются как `/article/{id}`, а не как отдельные ветви для каждого номера. Появилась автозагрузка классов.

2. Что делает роутер?

Роутер принимает метод и адрес запроса, сравнивает их с описанными маршрутами и вызывает нужный обработчик, передавая ему параметры из адреса. Если совпадений нет, роутер возвращает код 404.

3. Кто должен решать, что делать на `/article/{id}`?

Контроллер. Роутер лишь определяет, какому контроллеру и какому методу передать запрос, и передаёт ему значение `id`. Что именно показать для статьи с этим номером — решение контроллера, который берёт данные у модели и выбирает представление.

4. Что хранится в массиве `$routes` и что в нём хранится?

Двухуровневый массив: первым ключом идёт метод запроса (`GET` или `POST`), вторым — шаблон адреса, а значением — обработчик. Обработчик может быть замыканием или парой «класс контроллера и название метода», например `[ArticleController::class, 'index']`.

Вывод:

В ходе работы реализованы две ступени маршрутизации: ручной разбор адреса через `switch` с перенаправлением всех запросов на единую точку входа и полноценный класс `Router` с таблицей маршрутов, параметрами вида `{id}` и поддержкой методов запроса. Изучены принципы паттерна `MVC` и границы ответственности его компонентов.

Отдельно выявлена и исправлена ошибка исходного варианта: диагностический вывод выполнялся до разбора маршрута, из-за чего код 404 не устанавливался и страница «не найдена» отдавалась с кодом 200. Это показывает, почему в контроллере нельзя выводить данные до того, как определены заголовки ответа, — та же причина, по которой в дальнейшем вывод был вынесен в отдельный слой представлений.

Роутер получился расширяемым: добавление новых разделов не требует правки логики разбора адреса, что и проверено в следующих практиках, где к маршрутам добавились контроллеры, модели и представления.